University of Cape Town
Department of Computer Science

CSC3022F
Tutorial 1: A brief guide on Version Control, the C++ compilation process and submitting assignments

Feb, 2023

In this tutorial we will cover Git and version control, the C++ compilation build system and linking and, finally, how we expect you, as students of CSC3022F, to submit assignments and tutorials. In order to complete this tutorial you will require access to a machine running Ubuntu (or equivalent) such as those found in the senior computer laboratory. You will also need to download the 'Tutorial_1.tar.gz' file from the 'Tutorial 1' assignment tab on Vula.

1 Part One: Setting up a local repository

Last year you would have been introduced to the basics of version control and more specifically how to use Git. This tutorial will assume you understand the basics of Git (See `https://guides.github.com/introduction/git-handbook/` for a quick refresher). You will be expected to utilize version control for **all** CSC3022F tutorials and assignments. **Failure to submit a git repository will result in a significant penalty (usually -10%)**.

Lets go over the process of creating a local repository. If this is your first time using the machine, don't forget to configure your username and email address using the following two terminal commands:

```
git config --global user.name "Your name here"
git config --global user.email "Your email address here"
```

This information will appear in your commit history and will identify your work when working in a team. The next step is to extract the tarball and navigating to the exercise directory:

```
tar -xvf Tutorial_1.tar.gz
cd Tutorial_1
```

Within the folder you should find the following files:

```
ls
> driver.cpp factorial.cpp factorial.h
```

Now that we have all of the necessary files we can initialize our git repository:

```
git init
> Initialized empty Git repository in path/to/your/exercise/folder/ .git/
```

This creates a hidden .git folder inside the Tutorial folder. Make sure that this file exists by typing: *ls -a*. You should see the folder listed. **Take care not to delete this folder** because it contains your commit history. If this folder is deleted/corrupted it is as if you did not have a git repo to begin with.

Now lets add the included code to our git repo. If you type *git status* into the terminal you will see *driver.cpp, factorial.h and factorial.cpp* listed as untracked files. To track them we must first stage them by doing the following:

```
git add driver.cpp factorial.cpp factorial.h
```

Now if you type *git status* you should see the following:

```
> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
new file:   driver.cpp
new file:   factorial.cpp
new file:   factorial.h
```

This means that our files are staged and we can now commit them. Once our files have been committed, git can start tracking any modifications we make to the files. To commit the files, type the following:

```
git commit -m "Added tutorial files: driver.cpp, factorial.cpp and factorial.h"
```

You should then see the following output:

```
> git commit -m "Added tutorial files: driver.cpp factorial.cpp and factorial.h"
[main a9c4e9f] Added tutorial files: driver.cpp factorial.cpp and factorial.h
 3 files changed, 50 insertions(+)
 create mode 100644 Tutorial_1/driver.cpp
 create mode 100644 Tutorial_1/factorial.cpp
 create mode 100644 Tutorial_1/factorial.h
```

Note: Your terminal output may not look exactly the same (but it should be similar)

Now that our work has been committed we can briefly talk about commit scope and commit messaging. As most of you will already know, you can simply stage and commit all of your files at once and have a working git repo. We advise against this for a number of reasons. Firstly, it is completely unacceptable in a working environment and secondly, you aren't using git the way it was intended. This is where commit scope comes in. For example, say that you are tasked with creating the following image editing functions:

1. Convert an image to grayscale
2. Rotate an image
3. combine two images together

In terms of scope, these three functions are almost entirely distinct. Yes, you may create some Image base class that can be used by all of these functions but the functions are still independent from one another. In other words, these three functions are mutually exclusive and, as such, you should only be staging or committing work from one of them at a time.

If this is a bit confusing, a good way to tell if the scope of your current commit is too big is to think about the commit message you are going to type. Take a look at the following commit:

```
git commit -m "Added convert image to grayscale, rotate image and combine
image functionality. Fixed error where the image base class was not reading
the correct image dimensions and throwing an out of bounds exception"
```

As you can see this commit's scope is too big. It adds three new and independent functions and fixes a bug all in one commit. It would've been more appropriate to separate this into four distinct commits(1 for each feature and 1 for the bug fix). The message format you use is also something to take into consideration. Clearly saying "Added some files" is too vague but saying "Added convert image to grayscale function. I have no idea if it works because I spilled coffee over my laptop while trying to frantically finish this assignment" is way too long. Unfortunately commit scope and appropriate commit messages can be considered an art form typically guided by personal/institutional preference. While we do not expect you to be perfect when it comes to commit scope and messaging, **you can be penalized** for completely disregarding appropriate commit scope and messaging. Below are two really interesting articles on the issue (which you can read after the tutorial):

- How to write a git commit message: `https://chris.beams.io/posts/git-commit/`
- How to Write Good Commit Messages: A Practical Git Guide. `https://www.freecodecamp.org/news/writing-good-commit-messages-a-practical-guide/`

Now that we have a better understanding of commit scope, we can take a look at our git log by typing the following:

```
git log
```

You should see the following output:

```
commit a9c4e9f3d5fbd7f3f94a54779be3f53beaa7fd70 (HEAD -> main)
Author: Your name <your email>
Date:   [date of most recent commit]

    Added assignment files: driver.cpp factorial.cpp and factorial.h

commit 8b2900332110e114063e5d46bb99066ec4e49523 (origin/main, origin/HEAD)
Author: Your name <your email>
Date:   [date of initial commit]

    Initial commit
```

Note: Your terminal output may not look exactly the same (but it should be similar)

That's it! You have successfully made it through the first part of the tutorial and you should feel, at least somewhat, comfortable with staging and committing your own work. You can also store your code in an online repository like Github or Bitbucket. It makes working from multiple workstations incredibly easy and serves as an online backup of your work should your current piece of machinery decide to shed its mortal shell halfway through an assignment (it happens more often than people are willing to admit). The only caveat here is that **you need to ensure that your work is stored in a private repository**. If someone steals your code for an assignment, you can get disciplinary action taken against you (even if you did not consent or know that the person stole your work).

Lastly, it is recommended that you also look into *.gitignore* files (See `https://git-scm.com/docs/gitignore`). A *gitignore* is simply a file that allows you to list specific files or specific file formats that you never want to be included in your git repo. This is super useful for excluding unwanted binary files that are generated during the compilation process and by some IDEs (The aforementioned online repositories even have premade *.gitignore* files that you can automatically import when you create your repo).

2 Part 2: The Compilation Process

As you know, C++ is a compiled language whereby we convert our source code (The .cpp and .h files) into machine executable code. Under the hood this is actually a 3-step process:

1. Preprocessing
2. Compiling
3. Linking

Fortunately for us the a C++ compiler will combine steps 1 & 2 for us and, as a result, this tutorial will only cover steps 2 & 3 (just know that step is being done automatically). As a result we are first going to convert our source code into object code (Step 2) and then into machine executable code (Step 3).

In order to compile our code we need a compiler. The Lab machines use *gcc (version 9.3.0)* and we expect you to use it to. You can check your gcc version by typing *g++ –version* and you should see output that looks like follows:

```
> gcc --version
gcc (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

It is recommended that you make sure that your gcc version matches the Lab machines to avoid any compilation conflicts that may arise during assignment development. To compile our source files into object code, we type the following:

```
g++ -c factorial.cpp -o factorial.o -std=c++20
```

Note: Use *-std=c++2a* if using *GCC 9* and earlier.

Once that process has been completed you should see the factorial.o file in your current directory. The general formula for compiling a source file to an object file is:

```
g++ -c SOURCE_FILE -o TARGET_FILE -std=VERSION_NAME
```

In our case we have given our object files the '.o' extension but it can really be anything you want (with some exceptions). You'll also notice that we specified which version of C++ we want to use using the '-std' optional flag. For this course, we will be using C++ 20 and, as such, we expect you to use this flag to ensure that you are using the correct version of C++.

Lastly, you'll notice we didn't include the *factorial.h* header file in the compilation process despite the fact that we were compiling a factorial.o object file. If you open the *factorial.cpp* file you'll notice that one of the first lines is *#include factorial.h*. As you know from your notes, this is a preprocessor directive that is, in simple terms, the equivalent of copy pasting the *factorial.h* file into our *factorial.cpp* file during the preprocessing phase (Step 1) of the compilation process. It is incredibly unlikely that you'll ever be directly compiling a header file.

Now that we have compiled our *factorial.o* object file, go ahead and do the same for the *driver.cpp* file. Once you have completed this step you should have two separate object files for both the *driver* and *factorial* source files. Now we can move onto Linking (Step 3). As mentioned above this step of the compilation process involves us creating machine executable code using our object files. The linker takes our object files and matches any undefined symbols or method signatures to their implementations. This can be seen in the *driver.cpp* file where we import the *factorial.h* method signatures. These signatures

have no implementation and will throw an error if we trying to compile the *driver.cpp* by itself. This is where our *factorial.cpp* comes in since it contains our implementation of the method signatures specified in the *factorial.h* file. The linker's job is to match the undefined method signatures in *driver.cpp* to their implementation in *factorial.cpp*.

To link our code we use the following command:

```
g++ factorial.o driver.o -o driver.exe
```

Once this process completes you should see the *driver.exe* file in your current directory. To make sure that everything is running you should be able to execute this file using *./driver.exe* and you should see the following output (you may need to give the file execute permissions using *chmod*):

```
> ./driver
The factorial of 0 is: 1
The factorial of 1 is: 1
The factorial of 5 is: 120
The factorial of 13 is: 6227020800
```

That's it! You've successfully compiled and run your first C++ program. It can be a bit of an overwhelming process but it becomes a lot simpler when you think of it in terms of the steps mentioned above. However, doing this every time you want to run a your program will get tiresome rather fast. To solve this problem we will go over how to create makefiles for your C++ programs.

3 Part 3: Makefiles for your C++ programs

You would have encountered makefiles in second year courses. Fortunately Make and C++ compliment each other better than Make and Java do. For a quick refresher makefiles work on a target-source basis like follows:

```
TARGET: SOURCE_1 SOURCE_2 ... SOURCE_N
COMMAND_1
COMMAND_2
...
COMMAND_N
```

where the *TARGET* is the output file you want to produce, *SOURCE_1, SOURCE_2 and SOURCE_N* are the files or other targets needed to complete that process and *COMMAND_1, COMMAND_2 and COMMAND_N* are the commands needed to create our *TARGET* file. Using this information, we can create a makefile for our program that looks like so:

```
driver.exe: factorial.o driver.o
g++ factorial.o driver.o -o driver.exe -std=c++20

factorial.o: factorial.cpp
g++ -c factorial.cpp -o factorial.o -std=c++20

driver.o: driver.cpp
g++ -c driver.cpp -o driver.o -std=c++20
```

Now when you type *make* your makefile will create your *driver.exe* file automatically by first checking for *factorial.o* and *driver.o*. If either of them are missing it'll run the *factorial.o* or *driver.o* targets and create them (You may have to delete your current *driver.exe*, *factorial.o* and *driver.o* files to see this whole system at work). This makes compiling everything a lot easier and in the event that we change one of files, we can simply type *make* and the makefile will compile the appropriate files for us automatically (This is really useful for larger projects where we may not want to compile everything, every time we change some code).

Because we do not want students submitting binary files (**You can be penalized for this**), we should also create a *make clean* command that'll delete our *driver.exe* and *.o* files. To do this we simply add the following to our makefile:

```
clean:
rm *.o driver.exe
```

Now when you type *make clean* it should get rid of the executable and the object files. To finish things off you should go ahead and commit the makefile to your repo (With an appropriate commit message). Be careful to not include the binary files (.exe, .o, etc.) but if you have a *.gitignore* file you probably don't need to worry about it.

That concludes the tutorial!

1. Go ahead and submit the your work to Vula. Make sure to submit your work as a tarball and ensure that your git repository is included.
2. **(Optional):** As an additional task, try to make a generic makefile that is capable of producing a 'target' executable using any number of *.cpp* files. This may require you to do some additional reading but it will be hugely beneficial since you will never need to spend time worrying about the compilation process in future Tutorials and Assignments.
3. **(Optional):** Go through 'Part 4' of this tutorial.

4 Part 4: Getting the most out of your Assignments

Every year students go through an 'acclimatization' phase where the first one or two assignments result in students losing marks for unnecessary reasons. This section hopes to address that issue by giving you some *pointers* for completing assignments and tutorials.

4.1 Read the assignment's requirements:

Students will often lose marks for not reading/following an assignment's instructions. This ranges from huge errors like not including a git repo to simple errors like not separating a class' definition (.h) and implementation (.cpp). These mistakes add up quickly and can result in substantial penalties. In general all assignments and tutorials require the following:

- Using a Git Repository for Version Control
- Using a makefile for automated compilation. (A marker should simply type 'make' and the work should compile without issue.)
- A README detailing how your program works and instructions on how to run your program.
- That a student applied correct OOP principles when tackling the problem. (In C++, you can write incredibly complex programs without ever writing/instantiating a custom class/object. You will be penalized if you do this.)
- That a student's code is sufficiently commented and reasonably formatted (C++ code can get incredibly messy and hard to understand if a student disregards these two requirements. Understanding preprocessor macros/directives can greatly simplify your program) (*driver.cpp* uses a preprocessor directive to simplify the factorial function for example)

It is worth mentioning that these are general requirements. An assignment may ask you to do something atypical. You must still follow the assignment's instruction(s) and can/will be penalized for not doing so.

4.2 Make sure your work compiles on a Lab computer

When the tutors mark your assignments, they are instructed to use Lab computers. If your program does not compile and execute successfully on these machines, your assignment will be treated as if it does work correctly (this includes unexpected behaviour that is operating system specific). If you cannot test your work on a lab machine, you can 'ssh' into nightmare and compile your work there (nightmare should be running the same environment as those found on the lab computers). We do not typically make exceptions to this rule.

4.3 Make your code as easy to understand and use as possible

The tutors marking your assignments are human. They get tired, have off days and are capable of making mistakes. By making your Assignments as easy to understand and execute as possible, you reduce the risk of being marked incorrectly. The lecturers, TA and tutors are always willing to correct mistakes (this is why we have a one-week query period) but avoiding these mistakes is always preferable.

4.4 Don't be afraid to ask questions

As the progenitors of most of your practical work, the lecturers and TA are often unaware of the context at which you and your classmates approach a given assignment. If something seems incorrect or doesn't make sense, you can and should ask about it. You can make a post about it on the forums or, if you are uncomfortable doing that, you can send an email.

This goes hand in hand with choosing Class Representatives. It is their job to provide us with the aforementioned context when it comes to assignments, tests and general course content. The Course Convener may not always agree with everything the Class Representatives say or ask for but the potential for open discourse is always there.